5 b. Implement Hill climbing algorithm

```
def hill_climbing(graph, values, initial_state):

    if initial_state not in graph:
        print("Invalid initial state")
        return

    current = initial_state

    while graph[current]:

        best_neighbour = current

        for neighbour in graph[current]:

            if values[neighbour] > values[best_neighbour]:
                best_neighbour = neighbour

        if best_neighbour == current:
            break

        current = best_neighbour

    print("Final State:", current)
    print("Evaluation Value:", values[current])



# Read graph
graph = {}
```

```
n = int(input("Enter number of nodes: "))

for i in range(n):

    node = input("Enter node: ").strip()
    graph[node] = []

    e = int(input("Enter number of neighbours: "))

    for j in range(e):
        neighbour = input("Enter neighbour: ").strip()
        graph[node].append(neighbour)



# Read evaluation values
values = {}

print("\nEnter evaluation values:")

for node in graph:
    values[node] = int(input("Value of " + node + ": "))



# Multiple initial states
while True:

    print("\n--- HILL CLIMBING ---")

    initial_state = input("Enter initial state: ").strip()

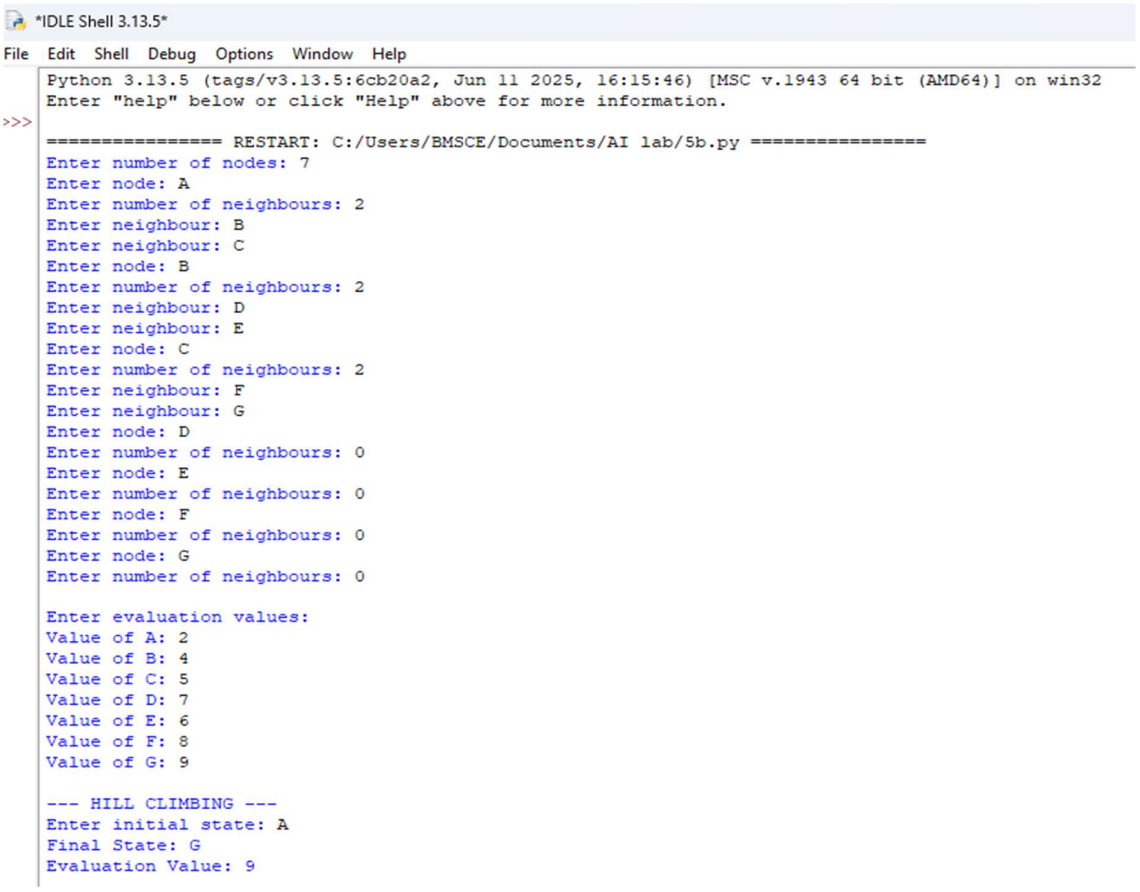
    hill_climbing(graph, values, initial_state)
```

```
    choice = input("\nDo you want to try again? (yes/no): ").strip().lower()

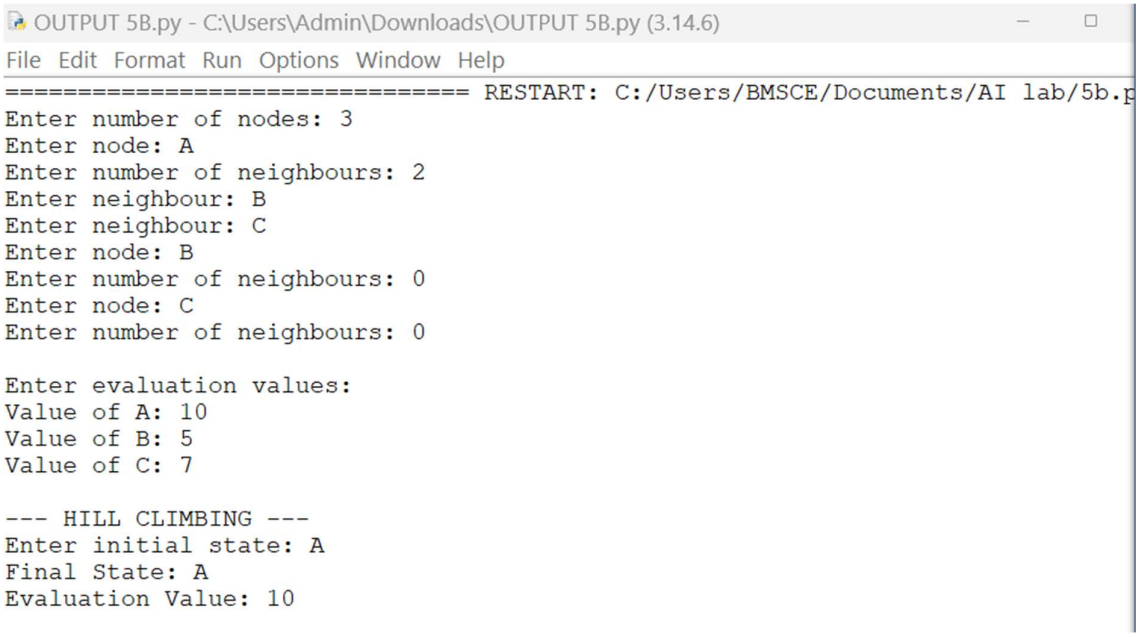

    if choice != "yes":
        print("Program terminated.")
        break
```

OUTPUT:

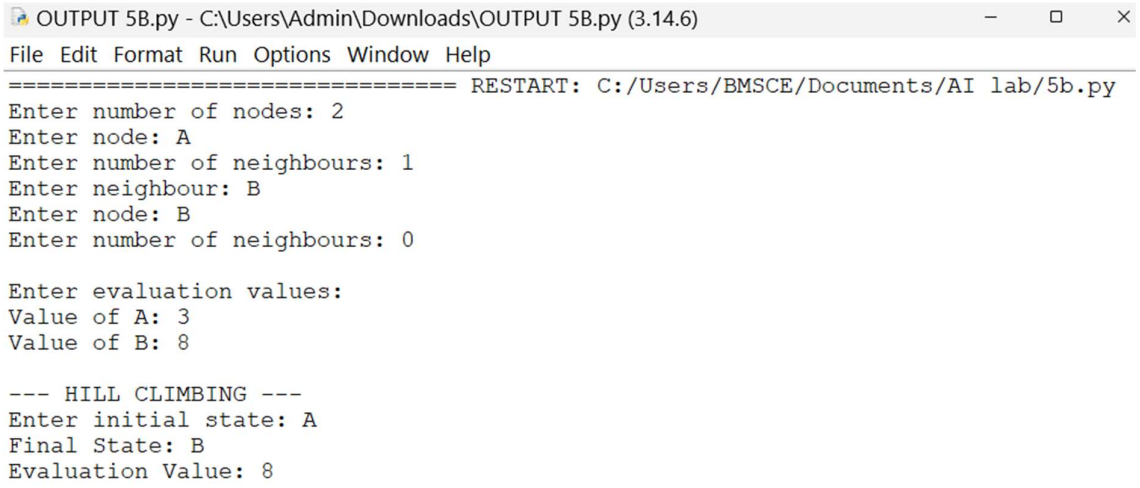
Test case 1 – Normal hill climbing



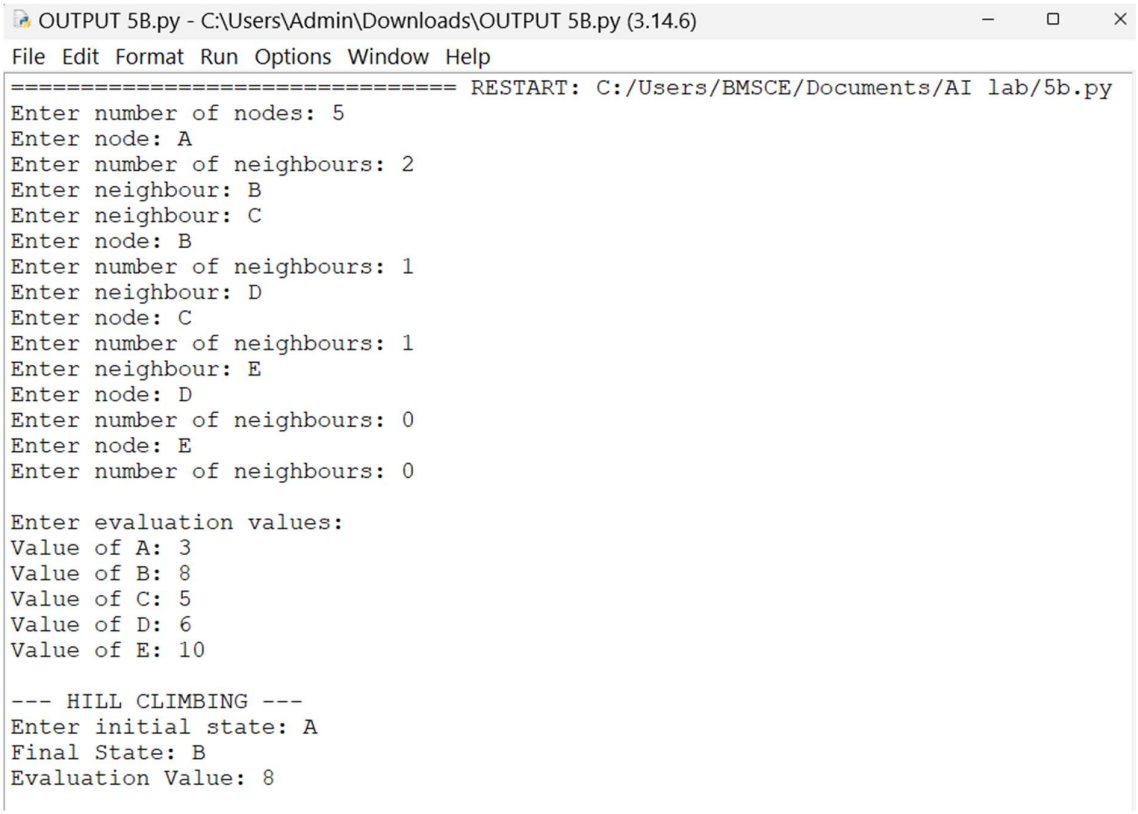
Test case 2 – Initial state is the best state



Test case 3 – Single neighbouring state



Test case 4 – Local maximum



Test case 5 – Plateau

```
========== RESTART: C:/Users/BMSCE/Documents/AI lab/5b.py
Enter number of nodes: 3
Enter node: A
Enter number of neighbours: 2
Enter neighbour: B
Enter neighbour: C
Enter node: B
Enter number of neighbours: 0
Enter node: C
Enter number of neighbours: 0

Enter evaluation values:
Value of A: 5
Value of B: 5
Value of C: 5

--- HILL CLIMBING ---
Enter initial state: A
Final State: A
Evaluation Value: 5
```

Test case 6 – All neighbouring states have lower values

```
================================ RESTART: C:/Users/BMSCE/Documents/AI lab/5b.py
Enter number of nodes: 3
Enter node: A
Enter number of neighbours: 2
Enter neighbour: B
Enter neighbour: C
Enter node: B
Enter number of neighbours: 0
Enter node: C
Enter number of neighbours: 0

Enter evaluation values:
Value of A: 9
Value of B: 6
Value of C: 4

--- HILL CLIMBING ---
Enter initial state: A
Final State: A
Evaluation Value: 9
```

Test case 7 – Multiple neighbouring states

```
================================ RESTART: C:/Users/BMSCE/Documents/AI lab/5b.py
Enter number of nodes: 4
Enter node: A
Enter number of neighbours: 3
Enter neighbour: B
Enter neighbour: C
Enter neighbour: D
Enter node: B
Enter number of neighbours: 0
Enter node: C
Enter number of neighbours: 0
Enter node: D
Enter number of neighbours: 0

Enter evaluation values:
Value of A: 2
Value of B: 5
Value of C: 9
Value of D: 7

--- HILL CLIMBING ---
Enter initial state: A
Final State: C
Evaluation Value: 9
```

Test case 8 – Initial state has no neighbours

```
================================ RESTART: C:/Users/BMSCE/Documents/AI lab/5b.py
Enter number of nodes: 1
Enter node: A
Enter number of neighbours: 0

Enter evaluation values:
Value of A: 5

--- HILL CLIMBING ---
Enter initial state: A
Final State: A
Evaluation Value: 5
```

Test case 9 – Invalid initial state

```
================================ RESTART: C:/Users/BMSCE/Documents/AI lab/5b.py
Enter number of nodes: 2
Enter node: A
Enter number of neighbours: 1
Enter neighbour: B
Enter node: B
Enter number of neighbours: 0

Enter evaluation values:
Value of A: 3
Value of B: 8

--- HILL CLIMBING ---
Enter initial state: X
Invalid initial state
```

Test case 10 – Local maximum versus global maximum

```
================================ RESTART: C:/Users/BMSCE/Documents/AI lab/5b.py
Enter number of nodes: 5
Enter node: A
Enter number of neighbours: 2
Enter neighbour: B
Enter neighbour: C
Enter node: B
Enter number of neighbours: 1
Enter neighbour: D
Enter node: C
Enter number of neighbours: 1
Enter neighbour: E
Enter node: D
Enter number of neighbours: 0
Enter node: E
Enter number of neighbours: 0

Enter evaluation values:
Value of A: 2
Value of B: 8
Value of C: 4
Value of D: 6
Value of E: 10

--- HILL CLIMBING ---
Enter initial state: A
Final State: B
Evaluation Value: 8
```